Hands on metabarcoding - Import and clean data

Francisco Pascoal

Objectives

- Illustrate a simple microbial ecology analysis, starting from the DADA2 output;
- Explain the R code used for all major steps:
  - data cleaning and normalization;
  - alpha diversity;
  - beta diversity;
  - taxonomy.

Organize your analyses (1)

You have the following folders:

- a folder with your R project;
- within, you have a /data and a /R folders.

```
# verify your directory
getwd()
# verify the files present
list.files("./data")
```

Organize your analyses (2)

We divided the downstream analysis in different scripts:

1. Hands_on_metabarcoding_Import_and_clean_data_exercises.R
2. Hands_on_metabarcoding_Alpha_diversity_exercises.R
3. Hands_on_metabarcoding_beta_diversity_exercises.R
4. Hands_on_metabarcoding_Taxonomy_exercises.R
5. Hands_on_metabarcoding_shared_microbiomes.R

Prepare your session (1)

The objective of the prepareSession script is to help you set up your overall analysis.

- List all the packages that you are going to use;
- Add some objects, or other settings you might need later on.

Prepare session (2)

To load a package into your R session, use the function library(name).

```
# load packages
____(dplyr) # grammar for data manipulation
____(tidyr) # create tidy data
____(stringr) # manipulate strings
____(ggplot2) # make plots
____(vegan) # diversity analyses
____(ulrb) # for some utils
____(readxl) # read excel files
____(gridExtra) # arrange ggplot2 plots
____(rstatix) # statistics tests
```

prepare session (2)

Solution:

```
# load packages
library(dplyr) # grammar for data manipulation
library(tidyr) # create tidy data
library(stringr) # manipulate strings
library(ggplot2) # make plots
library(vegan) # diversity analyses
library(ulrb) # for some utils
library(readxl) # read excel files
```

prepare session (3)

Additional things you can do to prepare your session.

```
# Vector with qualitative colors
qualitative_colors <-
  c("#E69F00", "#56B4E9", "#009E73",
    "#F0E442", "#0072B2", "#D55E00", "#CC79A7")
# quantitative scale
greens <- c("grey80", "#C7E9C0","#74C476","#238B45","#00441B")
# For reproducibility
set.seed(123)
```

Import and clean data (1)

Load the output from the upstream analysis.

```
# load ASV table
load("data/dada2_output")

# check variables stored in the environment
ls()
```

```
## [1] "def.chunk.hook" "seqtab.nochim"  "taxa"
```

Import and clean data (2) - Verify your ASV table

Useful functions:

- class() checks the class of an object;
- dim() checks the lenght of each dimension of an object;
- colnames() checks the names of columns.

```
# check class
____(seqtab.nochim)

# check size of rows and columns of seqtab.nochim
____(____)

# check column names
____(____)
```

Import and clean data (3) - Verify your ASV table

Solution:

```
# check class
class(seqtab.nochim)

# check size
dim(seqtab.nochim)

# check column names
colnames(seqtab.nochim)
```

Import and clean data (4) - Rarefaction

To normalize the total number of reads per sample, we will use rarefaction.

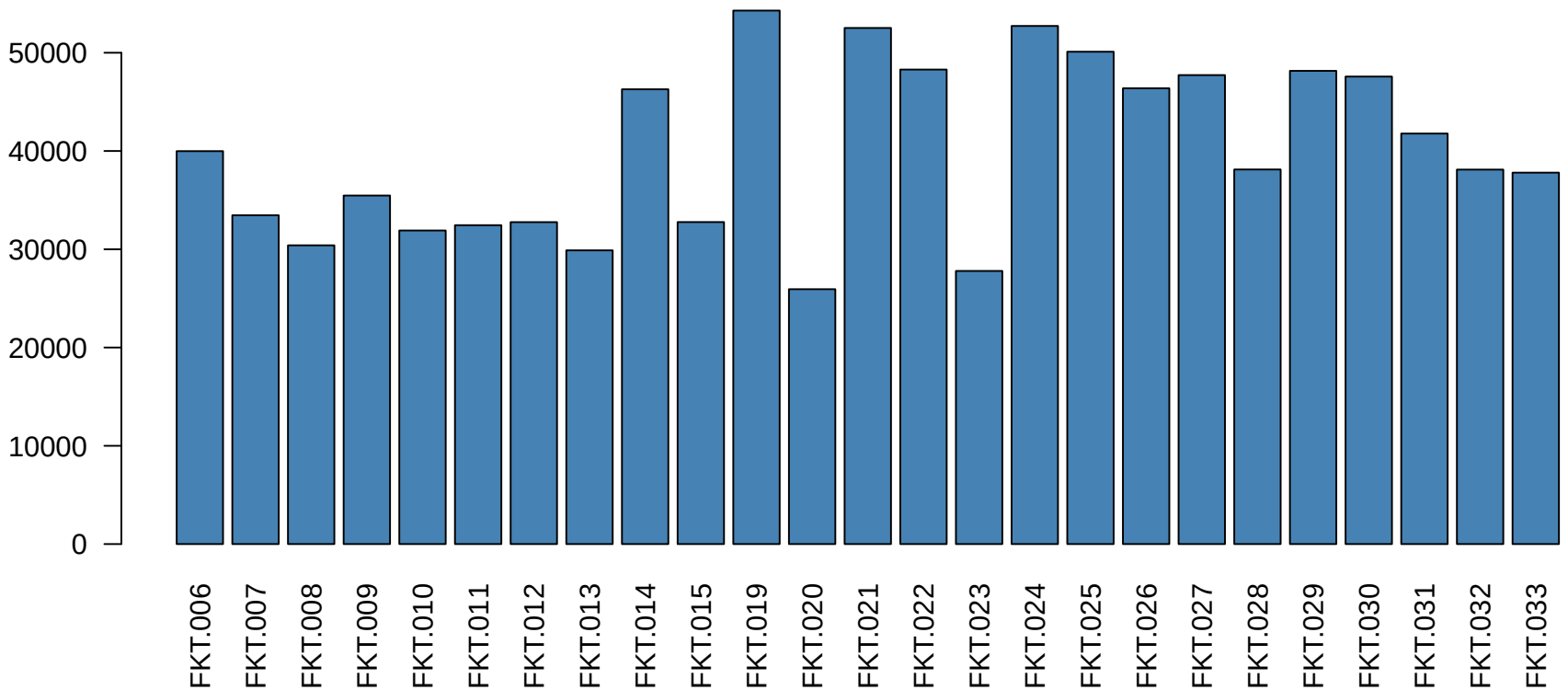
First, we must decide the rarefaction level.

```
# 1. check total reads per sample to decide rarefaction
# threshold
rowSums(____) %>%
  barplot(col = "steelblue", las = 2)
```

Import and clean data (5) - Rarefaction

Solution:

```
rowSums(seqtab.nochim) %>%
  barplot(col = "steelblue", las = 2)
```



Import and clean data (6) - Rarefaction

Compare some possible rarefaction levels.

Use the abline() function to add an horizontal line to the previous bar plot.

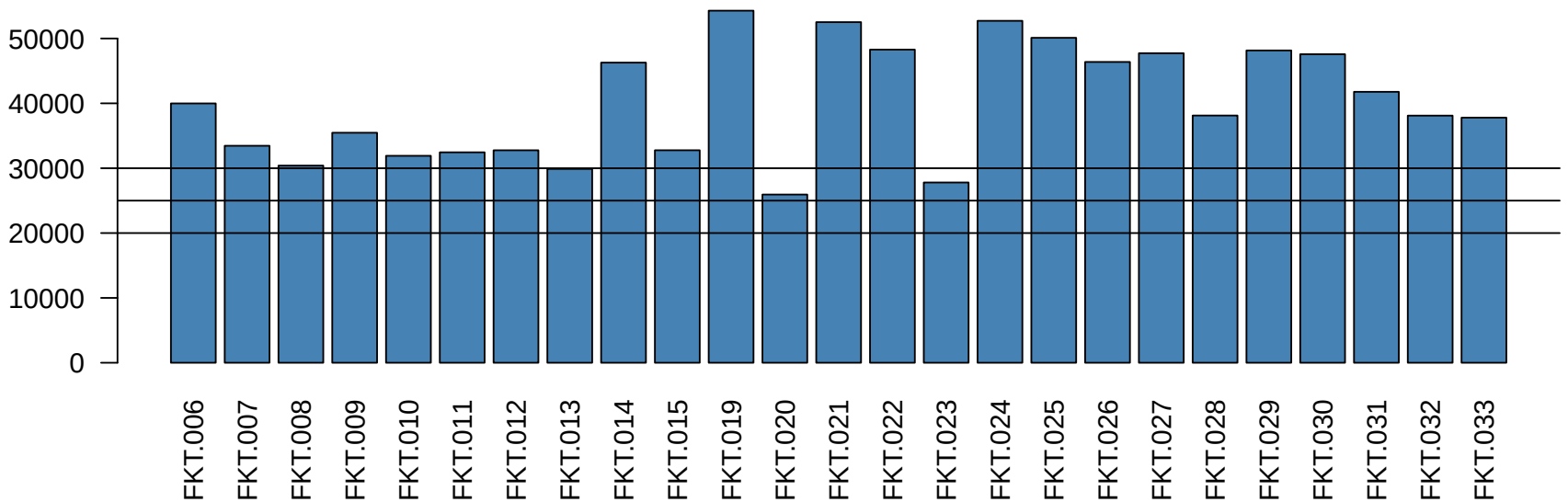
```
rowSums(seqtab.nochim) %>%
  barplot(col = "steelblue", las = 2)

# 2. Compare: 20 000, 25 000, and 30 000
abline(h = c(____, ____, ____))
```

Import and clean data (7) - Rarefaction

Solution:

```
rowSums(seqtab.nochim) %>%
  barplot(col = "steelblue", las = 2)
# 2. Compare: 20 000, 25 000, and 30 000 reads
abline(h = c(20000, 25000, 30000))
```



Import and clean data (8) - Rarefaction

Rarefy all samples to 25 000 reads per sample, using vegan Package.

```
# 3. Rarefy to 25000 reads per sample
set.seed(123); ASV_rarefied <- rrarefy(____, sample = ____)
```

Import and clean data (9) - Rarefaction

Solution:

```
# 3. Rarefy to 25000 reads per sample
set.seed(123); ASV_rarefied <-
  rrarefy(seqtab.nochim, sample = 25000)
```

Import and clean data (10) - Rarefaction

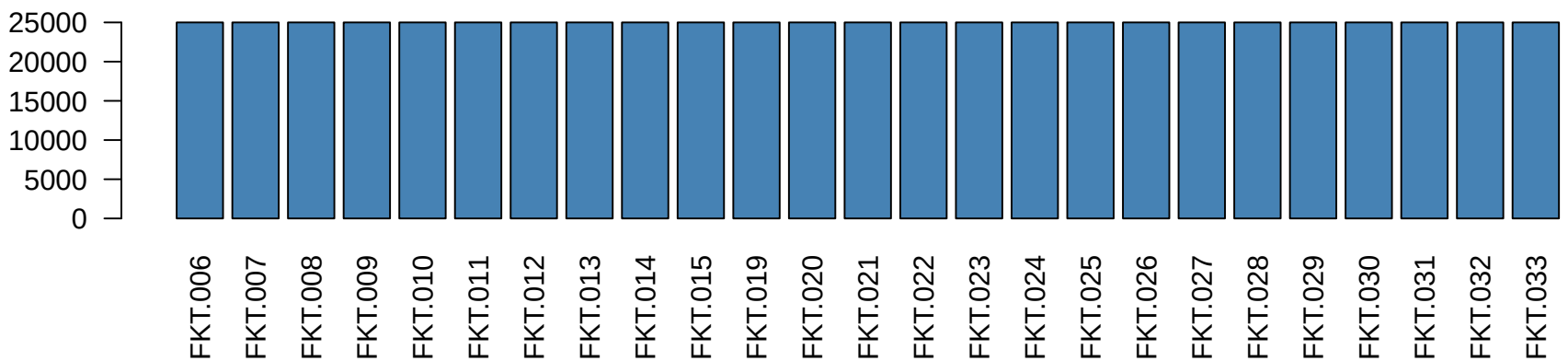
Repeat a bar plot to check if all samples have 25 000 reads.

```
# 4. Verify result
rowSums(____) %>%
  ____(col = "steelblue", las = 2)
```

Import and clean data (11) - Rarefaction

Solution.

```
# 4. Verify result
rowSums(ASV_rarefied) %>%
  barplot(col = "steelblue", las = 2)
```



Import and clean data (12) - more data cleaning

Start by transforming the ASV table from matrix to data frame, with samples as columns, so that it works better with pipes.

Useful functions:

- t() switches rows to columns.
- as.data.frame() transforms an object to data.frame.

```
# Turn ASV_rarefied to data.frame format
ASV_rarefied_df <- ____ %>%
  ____() %>% # switch rows to columns
  ____() # transform into data.frame
```

Import and clean data (13) - more data cleaning

Solution:

```
# Turn ASV_rarefied to data.frame format
ASV_rarefied_df <- ASV_rarefied %>%
  t() %>% # switch rows to columns
  as.data.frame() # transform into data.frame
```

Import and clean data (14) - more data cleaning

Put the sequences (in row names) in a column, and remove row names.

```
# Turn rownames into a column and then remove rownames
ASV_rarefied_df$Sequence <- rownames(____)
rownames(____) <- NULL
```

Import and clean data (15) - more data cleaning

Solution:

```
# Turn rownames into a column and then remove rownames
ASV_rarefied_df$Sequence <- rownames(ASV_rarefied_df)
rownames(ASV_rarefied_df) <- NULL
```

Import and clean data (16) - more data cleaning

Make an ID for each ASV in a new column, **ASV_ID**. The ASV_ID will have the following format: ASV_1, ASV_2, ...

Strategy: use the number of each row as unique identifier for each ASV.

Tips: the function paste0() can paste two strings togehter.

```
# The ASV ID can be in the form of ASV_1
ASV_rarefied_df$____ <- paste0("ASV_", rownames(____))
```

Import and clean data (17) - more data cleaning

Solution:

```
# The ASV ID can be in the form of ASV_1
ASV_rarefied_df$ASV_ID <- paste0("ASV_",
                                 rownames(ASV_rarefied_df))
```

Import and clean data (18) - view ASV table

You can check the ASV table with View(). Note that the column with sequences makes it hard to see information.

```
View(ASV_rarefied_df)
```

Import and clean data (19) - taxonomy table

Now we will get the taxonomic information for each ASV and join it to the ASV abundance table.

The taxonomic data is already stored in the object taxa.

```
taxa
```

Import and clean data (20) - taxonomy table

For the taxonomy output from DADA2, taxa, verify:

- size of each dimension;
- class of the object;
- column names.

```
# check dimension size
____(taxa)
# check class
____(____)
# check column names

____
```

Import and clean data (21) - taxonomy table

Solution:

```
# check dimension size
dim(taxa)
```

```
## [1] 14555     6
```

```
# check class
class(taxa)
```

```
## [1] "matrix" "array"
```

```
# check column names
colnames(taxa)
```

```
## [1] "Kingdom" "Phylum"  "Class"   "Order"   "Family"  "Genus"
```

Import and clean data (22) - taxonomy table

Put the sequences (in row names) in a new column, Sequence.

To do this, we need to transform the matrix into a data frame, then we can use the dplyr function, mutate(), to make a new column. Don't forget to remove the original row names, because they are no longer necessary.

Note: in dplyr notation, a single dot "." refers to data input.

```
ASV_taxa <- ____ %>%
  as.data.frame() %>%
  mutate(____ = rownames(.))
# remove row names
rownames(____) <- NULL
```

Import and clean data (23) - taxonomy table

Solution:

```
ASV_taxa <- taxa %>%
  as.data.frame() %>%
  mutate(Sequence = rownames(.))
# remove row names
rownames(ASV_taxa) <- NULL
```

Import and clean data (24) - merge ASV abundance and taxonomy

To merge data frames, the easiest option is to use join function from dplyr. In this case, we will use a **left_join()** function.

We will add the taxonomic information to the left of the ASV table, using the Sequence column as a reference.

```
# Merge the taxonomy and abundance tables
ASV_combined <- ASV_rarefied_df %>%
  ____(ASV_taxa, by = ____)
```

Import and clean data (25) - merge ASV abundance and taxonomy

Solution:

```
# Merge the taxonomy and abundance tables
ASV_combined <- ASV_rarefied_df %>%
  left_join(ASV_taxa, by = "Sequence")
```

Import and clean data (26) - taxa cleaning

Before advancing, we need to remove any undesirable taxonomic groups.

Start by checking the Kingdom level.

Note: the table() function makes a contingency table for each combination of factor levels.

```
# Overview Kingdom level
table(ASV_combined$____)
```

Import and clean data (27) - taxa cleaning

Solution:

```
# Overview Kingdom level
table(ASV_combined$Kingdom)
```

```
##
##  Archaea Bacteria
##     1391    13152
```

We don't have any eukaryotes, which is good, because we used 16S rRNA gene primers.

Import and clean data (28) - taxa cleaning

Even though we don't have eukaryotes, lets explicitly remove them, as well as any other undesirable groups.

Using dplyr functions, remove any:

- Eukaryotes;
- Chloroplasts;
- Mitochondria;
- NAs at Phylum level.

```
# Remove organelles, eukaryotes and NAs at phylum level, if any
ASV_combined_clean <- ____ %>%
  filter(____ != "Eukarya",
         Order ____ "Chloroplasts",
         Family != "____",
         !is.na(____))
```

Import and clean data (29) - taxa cleaning

Solution:

```
# Remove organelles, eukaryotes and NAs at phylum level, if any
ASV_combined_clean <- ASV_combined %>%
  filter(Kingdom != "Eukarya",
         Order != "Chloroplasts",
         Family != "Mitochondria",
         !is.na(Phylum))
```

Import and clean data (30) - taxa cleaning

Don't forget to verify your filtering result.

```
# verify if unwanted groups were removed
ASV_combined_clean %>% filter(Kingdom ____ "Eukarya")
ASV_combined_clean %>% filter(____ == "Chloroplasts")
ASV_combined_clean %>% filter(____ == "____")
ASV_combined_clean %>% filter(____(Phylum))
```

Import and clean data (31) - taxa cleaning

Solution:

```
# verify if unwanted groups were removed
ASV_combined_clean %>% filter(Kingdom == "Eukarya")
ASV_combined_clean %>% filter(Order == "Chloroplasts")
ASV_combined_clean %>% filter(Family == "Mitochondria")
ASV_combined_clean %>% filter(is.na(Phylum))
```

You must obtain **zero rows** for all filters.

Import and clean data (32) - metadata

Start by loading the metadata Excel file, using the **readxl** R package.

```
# Load metadata
metadata <- read_xlsx("./data/FKT_exp_amplicon_map.xlsx")
```

Import and clean data (33) - metadata

As always, start by checking your data. This time, use head() and str().

```
# see first five rows of metadata
head(____, n = 5)

# Use str() to see the structure of the metadata object
___(____)
```

Import and clean data (34) - metadata

Solution:

```
# See first five rows of metadata
head(metadata, n = 5)
```

```
## # A tibble: 5 x 7
##   SampleID    Experiment Treatment Replicate NGS_code SampleT
##   <chr>       <chr>      <chr>     <chr>     <chr>    <chr>
## 1 002_CTR_A   Sed002     CTR       A         FKT.001  DNA
## 2 002_0,015_A Sed002     Cd 0.015  A         FKT.002  DNA
## 3 002_0,15_A  Sed002     Cd 0.15   A         FKT.003  DNA
## 4 002_1,5_A   Sed002     Cd 1.5    A         FKT.004  DNA
## 5 002_15_A    Sed002     Cd 15     A         FKT.005  DNA
```

Import and clean data (35) - metadata

Solution (cont.):

```
# Use str() to see the structure of the metadata object
str(metadata)
```

```
## tibble [30 x 7] (S3: tbl_df/tbl/data.frame)
##  $ SampleID  : chr [1:30] "002_CTR_A" "002_0,015_A" "002_0,15
##  $ Experiment: chr [1:30] "Sed002" "Sed002" "Sed002" "Sed002"
##  $ Treatment : chr [1:30] "CTR" "Cd 0.015" "Cd 0.15" "Cd 1.5"
##  $ Replicate : chr [1:30] "A" "A" "A" "A" ...
##  $ NGS_code  : chr [1:30] "FKT.001" "FKT.002" "FKT.003" "FKT.
##  $ SampleType: chr [1:30] "DNA" "DNA" "DNA" "DNA" ...
##  $ Project   : chr [1:30] "FKt230602" "FKt230602" "FKt230602"
```

Import and clean data (36) - metadata

Some columns are redundant or provide unnecessary information.

Using **dplyr**, remove the columns:

- SampleID, SampleType and Project.

```
# Remove unnecessary columns
metadata.1 <- metadata %>%
  ____(-SampleID, ____, ____)
```

Import and clean data (37) - metadata

Solution:

```
# Remove unnecessary columns
metadata.1 <- metadata %>%
  select(-SampleID, -SampleType, -Project)
```

Import and clean data (38) - metadata

All variables are strings, but some should be factors.

Using dplyr, transform into factors the following variables:

- Experiment, Treatment, Replicate, NGS_code.

```
# Change to correct type
metadata.2 <- metadata.1 %>%
  mutate(Experiment = as.factor(Experiment),
         Treatment = factor(____,
                            levels = c("CTR", "Cd 0.015", "Cd 0.
         Replicate = as.factor(____),
         NGS_code = ____(____))
```

Import and clean data (39) - metadata

Solution:

```
# Change to correct type
metadata.2 <- metadata.1 %>%
  mutate(Experiment = as.factor(Experiment),
         Treatment = factor(Treatment,
                            levels = c("CTR", "Cd 0.015",
                                       "Cd 0.15", "Cd 1.5",
                                       "Cd 15")),
         Replicate = as.factor(Replicate),
         NGS_code = as.factor(NGS_code))
```

Import and clean data (40) - metadata

To verify if the changes were made, use the str() function.

```
str(metadata.2)
```

```
## tibble [30 x 4] (S3: tbl_df/tbl/data.frame)
##  $ Experiment: Factor w/ 2 levels "Sed002","Sed037": 1 1 1 1
##  $ Treatment : Factor w/ 5 levels "CTR","Cd 0.015",..: 1 2 3
##  $ Replicate : Factor w/ 3 levels "A","B","C": 1 1 1 1 1 2 2
##  $ NGS_code  : Factor w/ 30 levels "FKT.001","FKT.002",..: 1
```

Import and clean data (41) - metadata

It would be useful to have a column with the actual concentrations of Cadmium, instead of the treatment label.

Use **dplyr** and **stringr** to make a new column, Concentration, with Cadmium concentration.

Notes: In the Control group (CTR), Cd concentration was 0;

```
# Add column with concentration values
metadata.3 <- metadata.2 %>%
  mutate(Concentration = ifelse(Treatment == "CTR", 0,
                                str_remove(____, "Cd "))) %>%
  # make sure the concentration is numeric
  mutate(Concentration = as.double(____))
```

Import and clean data (42) - metadata

Solution:

```
# Add column with concentration values
metadata.3 <- metadata.2 %>%
  mutate(Concentration = ifelse(Treatment == "CTR", 0,
                                str_remove(Treatment, "Cd "))) %>%
  # make sure the concentration is numeric
  mutate(Concentration = as.double(Concentration))
```

Import and clean data (43) - metadata

The name NGS_code is confusing, change it to Sample, using base R.

```
# Change the name of NGS_code to Sample
colnames(metadata.3)[4] <- "____"
```

Import and clean data (44) - metadata

Solution:

```
# Change the name of NGS_code to Sample
colnames(metadata.3)[4] <- "Sample"
```

Import and clean data (45) - metadata

Also change the name of the object metadata.3 to metadata_clean.

```
# For simplicity change the name of the table
____ <- ____
```

Import and clean data (46) - metadata

Solution:

```
# For simplicity change the name of the table
metadata_clean <- metadata.3
```

Import and clean data (47) - final merge

Now that we have a clean metadata table, we can combine the previous ASV table, which includes abundance and taxonomy.

To be able to combine the metadata with the ASV table, you need to transform the ASV table (ASV_combined_clean) into long format, with a single column for Sample information.

Use **tidyr** and **dplyr** to make ASV_combined_clean a tidy object, call it ASVs_long.

```
____ <- ____ %>%
  pivot_longer(cols = rownames(seqtab.nochim),
               names_to = "Sample",
               values_to = "Abundance")
```

Import and clean data (48) - final merge

Solution:

```
ASVs_long <- ASV_combined_clean %>%
  pivot_longer(cols = rownames(seqtab.nochim),
               names_to = "Sample",
               values_to = "Abundance")
```

Import and clean data (49) - final merge

Print the first 3 rows of ASVs_long.

```
____(____, n = 3)
```

Import and clean data (50) - final merge

Solution:

```
head(ASVs_long, n = 3)
```

```
## # A tibble: 3 x 10
##   Sequence        ASV_ID Kingdom Phylum Class Order Family Gen
##   <chr>           <chr>  <chr>   <chr>  <chr> <chr> <chr>  <ch
## 1 TACGAAGGGTGCA~ ASV_1  Bacter~ Prote~ Gamm~ Xant~ Xanth~ Ste
## 2 TACGAAGGGTGCA~ ASV_1  Bacter~ Prote~ Gamm~ Xant~ Xanth~ Ste
## 3 TACGAAGGGTGCA~ ASV_1  Bacter~ Prote~ Gamm~ Xant~ Xanth~ Ste
```

The sample column is outside the margin.

Import and clean data (51) - final merge

Usually there are more than two ways of completing the same task.

Make ASV_combined_clean tidy, by using **ulrb** package:

```
# Alternative way, using ulrb package
ASVs_long <- ASV_combined_clean %>%
  prepare_tidy_data(sample_names = rownames(seqtab.nochim))
```

Import and clean data (52) - final merge

Now we can finally merge the metadata table (metadata_clean) with the ASV abundance and taxonomy information (ASVs_long).

Use **dplyr** to merge ASVs_long with metadata_clean, using the column Sample as a reference. Then, call the new object **ASVs_full**.

Recall: you can use left_join() to merge two data frames.

```
# Merge metadata_clean with ASVs_long
____ <- ____ %>%
  ____(____, by = ____)
```

Import and clean data (53) - final merge

Solution:

```
# Merge metadata_clean with ASVs_long
ASVs_full <- ASVs_long %>%
  left_join(metadata_clean, by = "Sample")
```

Import and clean data (54) - final cleaning step

Note that we have many zeros in the Abundance table, which represent absent ASVs in a given sample.

Now that we have our data well organized, it is trivial to remove the zeros from our dataset.

```
# Filter all values superior to zero in the Abundance column
ASVs_full <- ASVs_full %>%
  filter(____)
```

Import and clean data (55) - final cleaning step

Solution:

```
# Filter all values superior to zero in the Abundance column
ASVs_full <- ASVs_full %>%
  filter(Abundance > 0)
```